

Sprawozdanie z Projektu Podstawy Podejmowania Decyzji

Mieszko Czubiński, nr albumu: 284047

11 czerwca 2026

1 Wstęp

1.1 Opis

Badany problem jest trójwymiarowym problemem pakowania do kontenerów (ang. *bin packing problem*). Dodatkowo wprowadzono zależności między pakowanymi przedmiotami oraz uwzględniono ciągłe dostarczanie kolejnych produktów do rozpatrzenia. Celem jest minimalizacja liczby kontenerów potrzebnych do przewiezienia przedmiotów.

Rozwiązanie tego problemu może mieć zastosowanie przy załadunku towarów z fabryki do ciężarówek dostawczych. Dane użyte w przykładzie nawiązują do fabryki mebli. Produktem końcowym takiej fabryki są części mebli zapakowane w prostopadłościenną kartony. Ponieważ jeden mebel może być rozłożony na więcej niż jeden karton, wprowadzono między nimi zależności.

Fabryka nieustannie produkuje kolejne meble, dlatego istotne jest ciągłe odbieranie towaru przez ciężarówki. Są one stopniowo doładowywane napływającymi kartonami, gdy któraś jest dostatecznie zapełniona, zostaje wysłana w trasę i nie podlega już żadnym zmianom.

1.2 Obiekt podejmowania decyzji

Rozpatrywany system analizowany jest z perspektywy systemowej jako obiekt przetwarzający *decyzje* na *ocenę* tych decyzji.

Decydentem jest algorytm optymalizacyjny odpowiedzialny za załadunek w fabryce mebli, natomiast *przedmiotem decyzji* jest przydział kartonów z ciągle napływającego strumienia produkcji do kolejnych ciężarówek.

Wejściem (*decyzją*) jest przypisanie każdego kartonu do konkretnego kursu ciężarówki. *Zbiór decyzji dopuszczalnych* tworzą te przydziały, które jednocześnie:

- mieszczą się geometrycznie w wymiarach naczepy i nie kolidują ze sobą,
- nie przekraczają dopuszczalnej ładowności (limitu wagi),
- spełniają zależności między kartonami – karton może zostać załadowany tylko wtedy, gdy w tym samym kursie znajdują się wszystkie kartony, od których zależy.

Wyjściem (*oceną decyzji*) jest całkowita liczba ciężarówek użytych do przewiezienia całości towaru.

Zależność wyjścia od wejścia – sposób rozmieszczenia kartonów oraz reguły zależności określają, ile kartonów zmieści się w jednym kursie, a przez to – ile kursów potrzeba na obsłużenie całego strumienia produkcji. Kartony przydzielane są na bieżąco. Wysłanej ciężarówki nie można już zmienić.

1.3 Wymagania systemowe

Wymaganie na wyjście – kryterium decyzyjnym jest minimalizacja liczby użytych ciężarówek. Ze względu na NP-trudność problemu nie dąży się do gwarantowanego optimum, lecz do rozwiązania możliwie bliskiego optymalnemu.

Typ systemu – jest to *system automatycznego podejmowania decyzji*, w którym algorytm samodzielnie wyznacza plan załadunku przydzielanych kartonów, bez udziału człowieka.

2 Problem optymalizacyjny

Minimalizowana jest liczba ciężarówek (kursów) potrzebnych do przewiezienia wszystkich kartonów, przy zachowaniu ograniczeń przestrzennych, wagowych i zależnościowych.

2.1 Oznaczenia

- $I = \{1, \dots, n\}$ – zbiór kartonów; karton i : wymiary (w_i, h_i, d_i) , waga m_i ,
- $K = \{1, \dots, K_{\max}\}$ – zbiór dostępnych ciężarówek; naczepa: wymiary (W, H, D) , ładowność Q ,
- $\mathcal{D} \subseteq I \times I$ – zbiór zależności: $(i, j) \in \mathcal{D}$ oznacza, że karton i wymaga kartonu j w tym samym kursie.

2.2 Zmienne decyzyjne

$$y_k = \begin{cases} 1 & \text{ciężarówka } k \text{ jest użyta} \\ 0 & \text{w przeciwnym razie,} \end{cases} \quad x_{ik} = \begin{cases} 1 & \text{karton } i \text{ jedzie ciężarówką } k \\ 0 & \text{w przeciwnym razie,} \end{cases}$$

2.3 Funkcja celu

$$\min \sum_{k \in K} y_k$$

2.4 Ograniczenia

$$\sum_{k \in K} x_{ik} = 1, \quad \forall i \in I \quad (1)$$

$$x_{ik} \leq y_k, \quad \forall i \in I, k \in K \quad (2)$$

$$\sum_{i \in I} m_i x_{ik} \leq Q y_k, \quad \forall k \in K \quad (3)$$

$$x_{ik} \leq x_{jk}, \quad \forall (i, j) \in \mathcal{D}, k \in K \quad (4)$$

Ograniczenia (1) zapewniają, że każdy karton zostanie przewieziony dokładnie raz, (2) – kartony trafiają wyłącznie do użytych ciężarówek, (3) – nieprzekroczenie ładowności, a (4) – spełnienie zależności między kartonami.

Dodatkowo wymagany jest warunek nienakładania się kartonów przewożonych tą samą ciężarówką: dla każdej pary $i < j$ takiej, że $x_{ik} = x_{jk} = 1$, co najmniej jeden z prostopadłościanów leży w całości po jednej stronie drugiego wzdłuż którejś z osi.

3 Rozwiązanie

Trójwymiarowy problem pakowania z minimalizacją liczby kontenerów jest NP-trudny razem z dodatkowymi ograniczeniami (zależności, ciągły napływ kartonów oraz wysyłka aut w trybie online) wyklucza rozwiązanie dokładne. W celu rozwiązania problemu zastosowano **metaheurystykę** – *algorytm genetyczny*.

3.1 Reprezentacja permutacyjna i dekodery

Algorytm nie decyduje wprost, który karton trafi do której ciężarówki. Ustala jedynie **kolejność** π , w jakiej kartony są rozpatrywane przy załadunku – i to tę kolejność optymalizuje. Dla zadanej kolejności prosta procedura (*dekoder*) ładuje kartony jeden po drugim do ciężarówek, dając gotowy rozkład kartonów po autach:

- kartony rozpatrywane są w kolejności zadanej przez π ,
- kartony powiązane zależnościami tworzą *grupę*, która musi jechać tym samym autem; grupy wyznacza funkcja `build_groups`, łącząc ze sobą kartony zależne i pomijając te już wysłane; grupę ładuje się w całości albo wcale,
- każda grupa trafia do pierwszej ciężarówki, w której się mieści; jeśli nie mieści się w żadnej z otwartych – otwierana jest nowa,

Ułożenie kartonów wewnątrz auta wyznacza heurystyka *punktów ekstremalnych*. Oparta na zbiorze wolnych narożników, w które można wstawić karton. Karton trafia do najniższego i najgłębszego narożnika, w którym się mieści i nie nachodzi na inne kartony, po wstawieniu wzdłuż jego krawędzi powstają nowe naroża.

Zaletą oddzielenia kolejności od dekodera jest to, że algorytm nigdy nie tworzy załadunku łamiącego ograniczenia – przeszukuje tylko różne kolejności, a każda z nich daje poprawny plan.

```

def pack(order, gene_ids, by_id, groups, truck, departed=()):
    assigned = set(departed)
    bins = []
    for g in order:
        iid = gene_ids[int(g)]
        if iid in assigned:
            continue
        group = [by_id[c] for c in groups[iid]]
        if not any(b.add_group(group) for b in bins):
            new_bin = Bin(truck)
            new_bin.add_group(group)
            bins.append(new_bin)
        assigned.update(groups[iid])
    return bins

```

3.2 Funkcja przystosowania

Gdyby oceniać rozwiązanie wprost liczbą aut, wiele różnych kolejności miałoby identyczną ocenę i algorytm nie wiedziałby, które z nich są bliżej usunięcia kolejnego auta. Dlatego algorytm maksymalizuje ocenę, która nagradza auta wypełnione jak najpełniej:

$$F(\pi) = \frac{1}{m} \sum_{k=1}^m \left(\frac{V_k}{V} \right)^2,$$

gdzie m – liczba użytych aut, V_k – objętość zajęta w aucie k , a $V = W \cdot H \cdot D$ – objętość ładowni. Podniesienie do kwadratu nagradza rozkłady, w których jedno auto są pełne, a jedno prawie puste – takie najłatwiej domknąć, wyrzucając ostatnie, słabo zapełnione auto.

```

def fitness(order, items, truck, departed):
    groups = packer.build_groups(items, departed)
    gene_ids = [it.id for it in items]
    by_id = {it.id: it for it in items}
    bins = packer.pack(order, gene_ids, by_id, groups, truck,
        departed)
    capacity = truck.volume
    return sum((b.used_volume / capacity) ** 2 for b in bins) /
        len(bins)

```

3.3 Operatory genetyczne

- **Selekcja turniejowa** – losuje się t kolejności, a dalej przechodzi najlepsza z nich;
- **Mutacja przez zamianę** – w kolejności z prawdopodobieństwem p_m zamienia się miejscami dwa kartony;
- **Elityzm** – kilka najlepszych kolejności przechodzi do następnego pokolenia bez zmian.

3.4 Obsługa ciągłych dostaw

Co ustaloną liczbę pokoleń `ARRIVAL_INTERVAL` do każdej kolejności dopisywane są nowo przybyłe kartony, a ocena liczona jest już dla powiększonego zbioru. Obliczona wcześniej kolejność zostaje zachowana.

3.5 Odjazdy ciężarówek

Aby oddać cykl załadunku, ciężarówki wysyłane są w trakcie pracy algorytmu. Co `DEPART_INTERVAL` pokoleń najpełniejsze z otwartych aut zostaje wysłane: jego kartony trafiają do zbioru `departed` i nie biorą udziału w dalszych obliczeniach (dekoder je pomija). Żeby żaden karton nie został zapakowany dwa razy, wysłane kartony (`departed`) pomijane są i przy ładowaniu (`pack`), i przy tworzeniu grup zależności (`build_groups`).

```
if gen % config.DEPART_INTERVAL == 0 and bins:
    fullest = max(bins, key=lambda b: b.used_volume)
    shipped.append(fullest)
    departed_ids.update(fullest.contents)
    print(f"Truck departed: {len(fullest.contents)} boxes, {100*fullest.used_volume/truck.volume:.1f}% full")
```

Końcowy wynik to liczba aut wysłanych w trakcie symulacji powiększona o auta, które na końcu wciąż są doładowywane.

3.6 Parametry i instancja testowa

Jako ciężarówkę przyjęto realny lekki samochód dostawczy klasy o parametrach ładowni: $300 \times 180 \times 200$ cm (dł. $\times$ szer. $\times$ wys.) i ładowności 1500 kg. Symulacja startuje z 10 kartonami i co 10 pokoleń dostarcza po 5 nowych, co przy 200 pokoleniach daje 105 kartonów do rozwiezienia. Równolegle co 40 pokoleń wysyłana jest najpełniejsza ciężarówka.

Parametr	Wartość
Rozmiar populacji	50
Liczba pokoleń	200
Przedmioty początkowe	10
Interwał dostaw [pokolenia]	10
Liczność dostawy	5
Interwał odjazdów [pokolenia]	40
Ziaro generatora	42

Tabela 1: Parametry algorytmu genetycznego i symulacji dostaw.

4 Wyniki

5 Wnioski